

VECTORSKYTREE · SECURITY REVIEW

VectorSkyTree Escrow Review.

A security review of Escrow Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
VectorSkyTree	Escrow Review	Guardian	April 1 – April 2, 2026	April 9, 2026

§ 00

Contents.

20 pages — overview, confidence ranking, scope, 8 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 8 findings	P. 08
§ 05	Disclaimer	P. 19
§ 06	About Guardian	P. 20

§ 01

Overview.

Guardian reviewed Escrow Review across a single review round. The final report includes 3 Medium, 4 Low, 1 Informational findings.

AUDIT FIRM	METHODOLOGY
Guardian	Smart contract · Manual · Main Review
CLIENT	
VectorSkyTree	
PROJECT	
Escrow Review	
PREPARED BY	
Guardian	
AUDIT WINDOW	
April 1 – April 2, 2026	
FINAL REPORT	
April 9, 2026	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	3	0	3	0	0
Low	4	0	4	0	0
Informational	1	0	1	0	0
All severities	8	0	8	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Escrow Review.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Yearly increment change rewrites accrued cap	Logical Error	MEDIUM	ACKNOWLEDGED	p. 8
M-02	Min period change can extend live period	Logical Error	MEDIUM	ACKNOWLEDGED	p. 10
M-03	Same admin and claimant collapses controls	Logical Error	MEDIUM	ACKNOWLEDGED	p. 12
L-01	Unused period allowance is lost	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 13
L-02	Claimant cannot bound token output	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 14
L-03	Uniform action delay across risk levels	Configuration	LOW	ACKNOWLEDGED	p. 15
L-04	Fixed 24h price staleness assumption	Logical Error	LOW	ACKNOWLEDGED	p. 16
I-01	Bilateral recovery depends on both roles	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 17

MEDIUM **M-01**

ACKNOWLEDGED

Yearly increment change rewrites accrued cap

CATEGORY Logical Error
LOCATION Escrow.sol
REVIEW Main Review

DESCRIPTION

`executeYearlyIncrementChange` updates `yearlyIncrementUsd` immediately, but the contract only settles elapsed yearly growth later inside `_applyYearlyIncrements`. This means already-accrued time is repriced at the new increment rate when the next `claim` occurs instead of at the rate that was active while the time accrued.

```
function executeYearlyIncrementChange(uint256 _yearlyIncrementUsd) external onlyAdmin {
    _execute(keccak256(abi.encode("yearlyIncrement", _yearlyIncrementUsd)));
    yearlyIncrementUsd = _yearlyIncrementUsd; // new rate is stored immediately
    emit YearlyIncrementChanged(_yearlyIncrementUsd);
}

function _applyYearlyIncrements() internal {
    if (yearlyIncrementUsd == 0) return;
    uint256 yrs = (block.timestamp - lastIncrementTimestamp) / 365 days;
    if (yrs > 0) {
        maxPeriodUsd += yrs * yearlyIncrementUsd; // elapsed years use the latest rate
        lastIncrementTimestamp += yrs * 365 days;
        emit YearlyIncrementApplied(maxPeriodUsd, yrs);
    }
}
```

Impact:

- Raising `yearlyIncrementUsd` can retroactively over-credit the claimant for past years, which can be gamed by delaying `claim`.
- Lowering `yearlyIncrementUsd` can claw back claim capacity that had already accrued economically but was not yet materialized on-chain.
- Setting `yearlyIncrementUsd` to `0` can wipe out all previously accrued but unsettled yearly growth.

Example:

M-01 Yearly increment change rewrites accrued cap

CONTINUED

- `maxPeriodUsd` starts at `$1,000` and `yearlyIncrementUsd` starts at `$100`.
- Three full years pass without a `claim`, so under the original rate the cap would have grown to `$1,300`.
- Before the claimant makes the next `claim`, the admin executes `executeYearlyIncrementChange` with `$200`.
- The claimant waits until after that update to submit the delayed `claim`.
- `_applyYearlyIncrements` now applies `3 * $200`, so the cap becomes `$1,600` instead of `$1,300`.

The result is that the delayed config update changes the meaning of past elapsed time, not just future elapsed time.

— RECOMMENDATION

Before writing a new `yearlyIncrementUsd`, first settle any accrued growth under the old rate. One approach is to checkpoint the pending increase in `executeYearlyIncrementChange` before updating the variable. Another approach is to store an effective from timestamp so time before the change uses the old rate and time after the change uses the new rate.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Min period change can extend live period

CATEGORY Logical Error
LOCATION Escrow.sol
REVIEW Main Review

DESCRIPTION

`executeMinPeriodChange` updates `minPeriod` immediately, and `_rollPeriod` uses the current `minPeriod` when deciding whether the current claim window has expired. Because of that, increasing `minPeriod` can retroactively extend a period that would already have expired under the old setting.

```
function executeMinPeriodChange(uint256 _minPeriod) external onlyAdmin {
    if (_minPeriod < 7 days) revert MinPeriodTooShort();
    _execute(keccak256(abi.encode("minPeriod", _minPeriod)));
    minPeriod = _minPeriod; // applies to the current live period immediately
    emit MinPeriodSet(_minPeriod);
}

function _rollPeriod() internal {
    if (block.timestamp >= periodStart + minPeriod) {
        claimedThisPeriod = 0;
        periodStart = block.timestamp;
    }
}
```

Impact:

- The claimant can lose access to a fresh period even after the previous waiting window has already elapsed.
- The admin can delay the claimant's next withdrawal by extending `minPeriod` shortly before the claimant makes the first post-expiry `claim`.

Example:

M-02 Min period change can extend live period

CONTINUED

- The claimant uses the full `$1,000` budget with `minPeriod = 7 days`.
- After 7 days pass, the claimant should be eligible for a fresh period.
- Before the claimant calls `claim`, the admin executes `executeMinPeriodChange` to raise `minPeriod` to `30 days`.
- `_rollPeriod` now checks `periodStart + 30 days`, so the contract still treats the old period as active and the new `claim` reverts.

The result is that the delayed config update changes the status of the current period instead of only affecting future periods.

— RECOMMENDATION

Make `minPeriod` changes take effect only for the next period boundary. For example, keep the current live period under the old `minPeriod` and apply the new value only after `_rollPeriod` starts a new period. Another option is to settle the current period at execution time and then start a new period using the new duration.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Same admin and claimant collapses controls

CATEGORY Logical Error
LOCATION Escrow.sol
REVIEW Main Review

DESCRIPTION

The constructor does not prevent `admin` and `claimant` from being set to the same address. When that happens, the contract's two-party approval model degrades because `_callerBit` returns `ADMIN_BIT` first, so the shared address can never contribute `CLAIMANT_BIT`.

```
function _callerBit() internal view returns (uint8) {  
    if (msg.sender == admin) return ADMIN_BIT;  
    if (msg.sender == claimant) return CLAIMANT_BIT;  
    return 0;  
}
```

As a result, express role-change execution cannot be reached because `roleApprovals[hash]` can never become `BOTH_BITS`. Similarly, `cancelAction` can never fully cancel a pending action because `cancelApprovals[hash]` can never reach `BOTH_BITS`.

RECOMMENDATION

Reject deployments where `admin == claimant`, or handle the shared-role case explicitly.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Unused period allowance is lost

CATEGORY Unexpected Behavior
LOCATION Escrow.sol
REVIEW Main Review

DESCRIPTION

The contract does not carry forward unused allowance from one period to the next. When `_rollPeriod` detects that the old period has expired, it resets `claimedThisPeriod` to `0` and starts a new period at the current timestamp. Any unclaimed portion of `maxPeriodUsd` from the prior period is discarded.

```
function _rollPeriod() internal {  
  if (block.timestamp >= periodStart + minPeriod) {  
    claimedThisPeriod = 0; // prior-period usage is cleared with no rollover accounting  
    periodStart = block.timestamp;  
  }  
}
```

This behavior may be intentional, but a claimant could reasonably expect missed allowance to accumulate or at least remain claimable until explicitly forfeited. So if the claimant forgets to call `claim` during a period, the unused portion of that period's budget is lost.

RECOMMENDATION

Document this behavior explicitly. If rollover is desired, add separate accounting for unspent allowance rather than resetting only `claimedThisPeriod`.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Claimant cannot bound token output

CATEGORY Unexpected Behavior
LOCATION Escrow.sol
REVIEW Main Review

DESCRIPTION

`claim` lets the claimant choose a `_usdAmount`, but it does not let them enforce a minimum acceptable token amount.

```
function claim(address _token, uint256 _usdAmount) external onlyClaimant {  
    ...  
    uint256 tokenAmount = _usdToTokens(_usdAmount, cfg); // amount is fixed only at execution  
    time  
    claimedThisPeriod += _usdAmount;  
    IERC20(_token).safeTransfer(claimant, tokenAmount);  
    emit Claimed(claimant, _token, tokenAmount, _usdAmount);  
}
```

Example:

- The claimant checks `getTokenAmount` and sees that claiming `$1,000` should return `0.5 WETH`.
- Before the transaction is mined, the oracle price updates upward (or receives a bad oracle update).
- `claim` executes using the new higher price, so the claimant receives only `0.45 WETH`.
- The full `$1,000` is still counted against `claimedThisPeriod`.

RECOMMENDATION

Extend `claim` to accept a claimant-specified minimum token amount, for example `minTokenAmount`, and revert when the oracle-priced output falls below that floor.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Uniform action delay across risk levels

CATEGORY	Configuration
LOCATION	Escrow.sol
REVIEW	Main Review

DESCRIPTION

The contract uses a single `ACTION_DELAY` for all delayed operations, including role changes, emergency withdrawals, and routine configuration updates. This keeps the implementation simple, but it also means actions with very different risk profiles all share the same review window.

For example, `signalAdminChange`, `signalWithdraw`, and `signalPriceFeedChange` all mature after the same `7 days`. In practice, a bad role change or withdrawal is much more security-sensitive than a routine oracle update. A single delay forces the system to choose one compromise between security response time and operational flexibility.

As a result,

- High-impact actions such as role changes and `withdraw` do not get a longer reaction window than routine parameter updates.
- Operational changes such as price feed or cap updates cannot be expedited without also shortening the delay for more dangerous actions.
- The timelock cannot be tuned per action type to match the actual security and operational needs of the escrow.

RECOMMENDATION

Consider separating delays by action class. A common split is to use a longer delay for role changes and emergency withdrawals, and a shorter delay for routine configuration changes such as feed or cap updates.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Fixed 24h price staleness assumption

CATEGORY	Logical Error
LOCATION	Escrow.sol
REVIEW	Main Review

DESCRIPTION

`_getPrice` hardcodes a single `24 hours` freshness window for every configured Chainlink feed. That assumes all supported assets share the same acceptable staleness threshold, but in practice feeds can have different heartbeats.

```
function _getPrice(TokenConfig memory cfg) internal view returns (uint256) {
    (, int256 answer,, uint256 updatedAt,) = cfg.priceFeed.latestRoundData();
    if (answer <= 0) revert InvalidPrice();
    if (block.timestamp - updatedAt > 24 hours) revert StalePrice(); // fixed threshold for every
    feed
    return uint256(answer);
}
```

As a result, feeds whose expected freshness is shorter than `24 hours` can still be accepted even after they should be treated as stale, allowing `claim` to settle against outdated prices. Similarly, feeds whose heartbeat is longer than `24 hours` can become unusable even when the oracle is behaving as configured, blocking otherwise valid claims.

RECOMMENDATION

Store a per-token maximum price age alongside each feed configuration and check `updatedAt` against that token-specific value instead of a universal `24 hours`. If the escrow is intended for L2 deployment, also consider gating claims on the relevant sequencer uptime feed.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Bilateral recovery depends on both roles

CATEGORY Warning
LOCATION Escrow.sol
REVIEW Main Review

DESCRIPTION

The escrow treats `admin` and `claimant` as equal co-signers for recovery and cancellation. Either role can initiate replacement of the other after `ACTION_DELAY`, and `cancelAction` requires approvals from both roles. As a result, the contract's safety model is bilateral: the `claimant` is not only a capped beneficiary, but also a participant in privileged recovery flow.

```
function cancelAction(bytes32 actionHash) external onlyAdminOrClaimant {  
    cancelApprovals[actionHash] |= _callerBit();  
    if (cancelApprovals[actionHash] == BOTH_BITS) {  
        delete pendingActions[actionHash]; // both roles must participate  
    }  
}
```

In a key-compromise scenario, `admin` and `claimant` must work together to cancel hostile pending actions before execution. `ACTION_DELAY` provides notice and time to coordinate, but it does not create a unilateral admin-side veto.

For example:

- An attacker uses a leaked `claimant` key to call `signalAdminChange(attackerAdmin)`.
- The honest `admin` cannot clear that action alone, because `cancelAction` requires both role approvals.
- If the honest `claimant` still has practical control of the leaked key, the `admin` and `claimant` can coordinate to cancel the action before `ACTION_DELAY` expires.
- If they fail to coordinate in time, `attackerAdmin` can execute the rotation after the delay.

RECOMMENDATION

Document the trust assumptions explicitly: `claimant` is an equal co-signer in recovery and cancellation flows. Operationally, both roles should be prepared to act together if a key is leaked, because hostile pending actions cannot be cancelled by one side alone. If

I-01 Bilateral recovery depends on both roles

CONTINUED

this bilateral model is not desired, recovery powers should be concentrated on an admin-side multisig instead of being shared with the claimant.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.